



UNIVERSIDAD INTERNACIONAL DEL ECUADOR

FACULTAD DE INGENIERÍAS DIGITALES Y TECNOLOGÍAS
EMERGENTES

ESCUELA DE INGENIERÍA EN INTELIGENCIA ARTIFICIAL

Generación de Diagramas funcionales y Arquitectura de Software del Juego
“Piedra, Papel o Tijera”

Autor: David Rafael Dueñas Ripalda

Docente Tutor: Mgs. Estefanía Vanessa Heredia Jiménez

Asignatura: LOGICA DE PROGRAMACION 1-SIN-1B

Ambato, Ecuador

2026

REINVENTEMOS
EL FUTURO

ÍNDICE TEMÁTICO

1. INTRODUCCIÓN
2. DESCRIPCIÓN DEL PROBLEMA
3. RELACIÓN CON LOS CONTENIDOS DE LA ASIGNATURA
4. EXPLICACIÓN DEL SISTEMA DESARROLLADO
 - 4.1 Arquitectura del Software (Modelo-Vista-Controlador)
 - 4.2 Mapeo y Trazabilidad de Requerimientos Funcionales
 - 4.3 Desglose Algorítmico y Trazabilidad de Código mediante Flujoramas Especializados
5. REFLEXIÓN SOBRE EL IMPACTO DE LA TECNOLOGÍA EN LA SOCIEDAD
6. CONCLUSIONES Y LOGROS RELACIONADOS
7. IMPLICACIONES Y LIMITACIONES DEL PROYECTO
8. REFERENCIAS BIBLIOGRÁFICAS

1. INTRODUCCIÓN

En el panorama contemporáneo de las ciencias de la computación y la ingeniería de sistemas, el desarrollo de soluciones informáticas ha trascendido la mera automatización de tareas operativas aisladas, constituyéndose actualmente como el eje estructural y regulador de la toma de decisiones, la interconectividad y la interacción humana en múltiples niveles de la sociedad. Siguiendo las directrices y fundamentos clásicos de la ingeniería de software, el diseño e implementación de cualquier aplicativo computacional, independientemente de su escala lúdica o industrial, demanda un rigor formal absoluto que vincule estrechamente la conceptualización teórica de los diagramas abstractos con la ejecución práctica de un código fuente modular, legible y eficiente.

Este informe técnico documenta de manera exhaustiva el diseño integral, la arquitectura sistémica y la fundamentación científica del Proyecto Integrador correspondiente a la asignatura de Lógica de Programación. El sistema desarrollado consiste en una plataforma interactiva y automatizada basada en el juego clásico "Piedra, Papel o Tijera". Lejos de constituir un simple ejercicio básico de codificación secuencial, este proyecto se ha estructurado bajo los más altos estándares profesionales de desarrollo, sintetizando las competencias clave de modularización, encapsulamiento de datos, control de flujos concurrentes y toma de decisiones estocásticas. El núcleo del software ha sido diseñado con un enfoque orientado a objetos que garantiza la total separación de responsabilidades lógicas, proveyendo un entorno controlado y auditable para estudiar la interacción simbólica entre un agente humano y un oponente artificial gobernado por leyes probabilísticas uniformes.

2. DESCRIPCIÓN DEL PROBLEMA

Las dinámicas de comunicación, trabajo y resolución de conflictos en la sociedad actual se encuentran masiva y profundamente mediadas por plataformas y ecosistemas digitales. Sin embargo, un desafío crítico dentro del desarrollo ágil de software a nivel industrial es la tendencia recurrente a ignorar el modelado arquitectónico previo, incurriendo en malas prácticas de codificación como el acoplamiento severo e indiscriminado entre la lógica interna de negocio y las interfaces visuales o de consola del usuario periférico (código monolítico o "código espagueti"). Este fenómeno degrada severamente la mantenibilidad del sistema, restringe su escalabilidad tecnológica, dificulta la implementación de pruebas unitarias automatizadas e incrementa exponencialmente los costos económicos y de tiempo en fases de mantenimiento evolutivo.

Para abordar esta problemática metodológica dentro del contexto formativo de la Ingeniería en Inteligencia Artificial, se planteó el diseño formal de un software para el juego "Piedra,

REINVENTEMOS
EL FUTURO

Papel o Tijera". Aunque las reglas operativas de este juego son de dominio público y fácil comprensión, su traducción a un sistema computacional plantea retos lógicos y arquitectónicos sustanciales: la sincronización de hilos de entrada/salida de datos de manera síncrona, la persistencia temporal y control de mutabilidad del estado global de la sesión (marcador acumulativo) y la formulación de un subsistema de toma de decisiones para la computadora que mitigue por completo la presencia de patrones predictivos explotables por el jugador. Por lo tanto, el problema central radica en construir un núcleo algorítmico autónomo que garantice la equidad de condiciones mediante una distribución de probabilidad perfectamente uniforme, permitiendo al mismo tiempo la migración transparente del aplicativo hacia interfaces web o móviles sin necesidad de modificar una sola línea de la lógica de negocio central.

3. RELACIÓN CON LOS CONTENIDOS DE LA ASIGNATURA

El diseño, modelado algorítmico y codificación del presente sistema informático representa una manifestación empírica directa y consolidada de las cuatro unidades temáticas fundamentales que estructuran el contenido programático de la asignatura de Lógica de Programación. A continuación, se detalla la integración disciplinar de dichas competencias teóricas dentro del software desarrollado:

- Estructuras de Control Condicionales (Unidades 1 y 2): Se aplicaron bloques condicionales anidados y estructuras de selección múltiple estructurada (switch-case / if-elif-else) para mapear matemáticamente la matriz de pagos determinista del juego. Esto permite contrastar de manera simultánea la jugada provista por el usuario contra la acción generada por la computadora, evaluando la lógica booleana de las variables para resolver y declarar de forma inequívoca estados de victoria, derrota o empate en cada ronda.
- Estructuras Repetitivas y Control de Ciclos (Unidad 3): La persistencia y continuidad de la sesión activa del juego se sostiene operativamente mediante la implementación de un bucle de control cíclico condicionado (mientras / while). Esta estructura gobierna el flujo principal del programa, permitiendo la ejecución indeterminada y sucesiva de múltiples rondas de juego, y manteniendo la integridad del marcador general hasta que el usuario invoque explícitamente la instrucción de salida del sistema.
- Modularización y Programación Orientada a Objetos (Unidad 4): El sistema rechaza de raíz la ejecución secuencial monolítica, distribuyendo las responsabilidades operativas en métodos especializados y clases independientes con alta cohesión y bajo acoplamiento. A través del encapsulamiento, se resguarda el estado de los contadores internos del marcador, impidiendo la manipulación externa de los datos fuera de los canales autorizados del controlador.

4. EXPLICACIÓN DEL SISTEMA DESARROLLADO

4.1 Arquitectura del Software (Modelo-Vista-Controlador)

Con el firme propósito de dar un cumplimiento estricto e irrestricto al requerimiento no funcional de acoplamiento débil y extensibilidad de componentes, la solución informática se estructuró adoptando rigurosamente el patrón arquitectónico Modelo-Vista-Controlador (MVC). Esta segmentación aísla por completo las funciones operativas del sistema en tres capas lógicas autónomas:

- Capa de Dominio (Modelo): Compuesta por módulos lógicos independientes como GameEngine, Scoreboard, Player y CPUOpponent. Su función es almacenar el estado del juego, las variables acumulativas del marcador y administrar la lógica probabilística de la CPU. El oponente automatizado utiliza un Generador de Números Pseudoaleatorios (PRNG) que garantiza una distribución de probabilidad uniforme elemental ($P = 1/3$), emulando con exactitud el Equilibrio de Nash de la teoría de juegos para asegurar un entorno interactivo justo y auditable.
- Capa de Presentación (Vista): Representada por componentes de interfaz como ConsoleInterface e InputScanner. Su responsabilidad única y exclusiva radica en canalizar los flujos de Entrada/Salida (E/S), desplegar los menús de texto y capturar los comandos crudos del usuario, abstrayéndose por completo de cómo se procesan o evalúan los datos.
- Capa de Control (Controlador): Centralizada en la clase GameController. Actúa como el puente intermedio y orquestador del sistema; intercepta los estímulos y datos capturados por la Capa de Presentación, invoca los métodos correspondientes en la Capa de Dominio para modificar el estado del juego y coordina la actualización inmediata de la interfaz visual del usuario.

4.2 Mapeo y Trazabilidad de Requerimientos

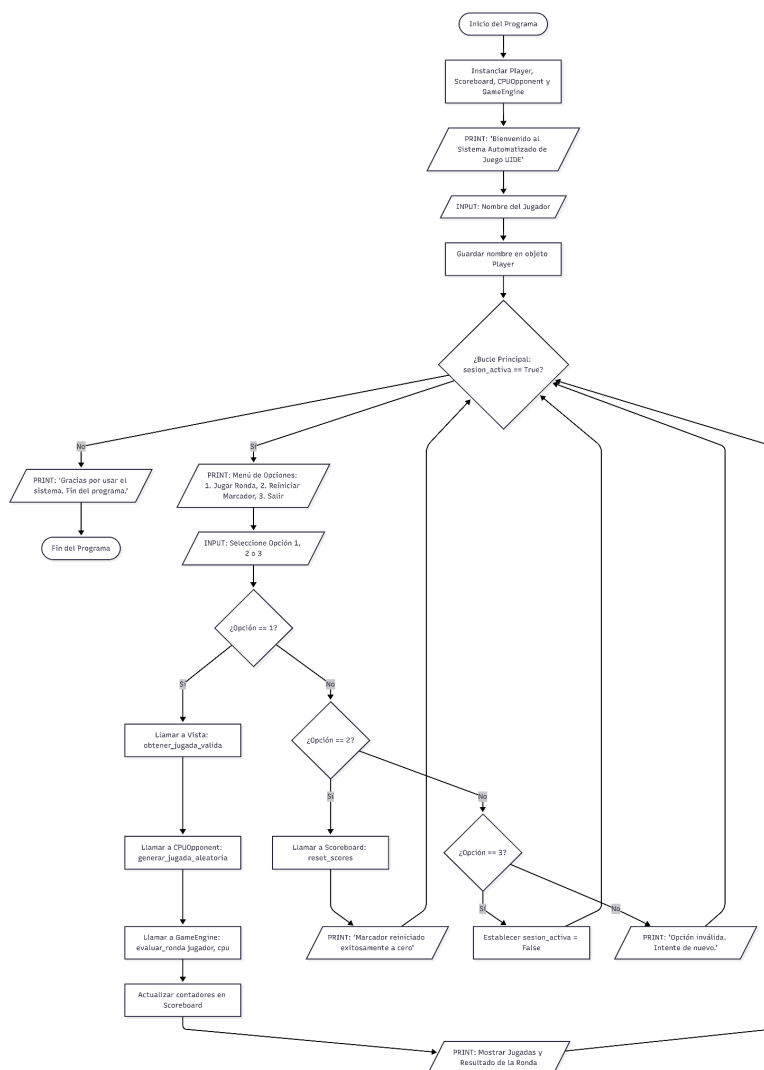
Para demostrar científicamente la viabilidad, cobertura e integridad de la solución implementada, se generó una matriz de trazabilidad técnica que vincula cada requerimiento de ingeniería de software con los componentes y clases específicos desplegados en el repositorio oficial de GitHub del proyecto:

Código Req.	Descripción del Requerimiento de Software	Componente Técnico Responsable en Código
RF-01	El sistema debe capturar y registrar un identificador de texto provisto por el usuario para la personalización de la interfaz.	Clase Player (Encapsulamiento de atributo Nombre)
RF-02	El sistema debe restringir y validar la entrada del teclado, admitiendo únicamente jugadas lícitas (Piedra, Papel, Tijera).	Componente InputScanner / Validación en Capa Vista
RF-03	Generación estocástica de la jugada del computador mediante una distribución equiprobable que mitigue patrones predictivos.	Clase CPUOpponent (Subsistema PRNG matemático)
RF-04	Evaluación determinista de las jugadas para dictaminar de forma inequívoca estados de victoria, derrota o empate por ronda.	Método evaluar_ronda() en la clase GameEngine
RF-05	Actualización e incremento continuo en memoria local del marcador general acumulativo de la sesión activa de juego.	Clase Scoreboard (Métodos incrementales mutadores)
RF-06	Función explícita accesible desde el menú para restablecer los contadores del marcador general de partidas a cero.	Método mutador reset_scores() en clase Scoreboard

4.3 Desglose Algorítmico y Trazabilidad de Código mediante Flujoramas Especializados

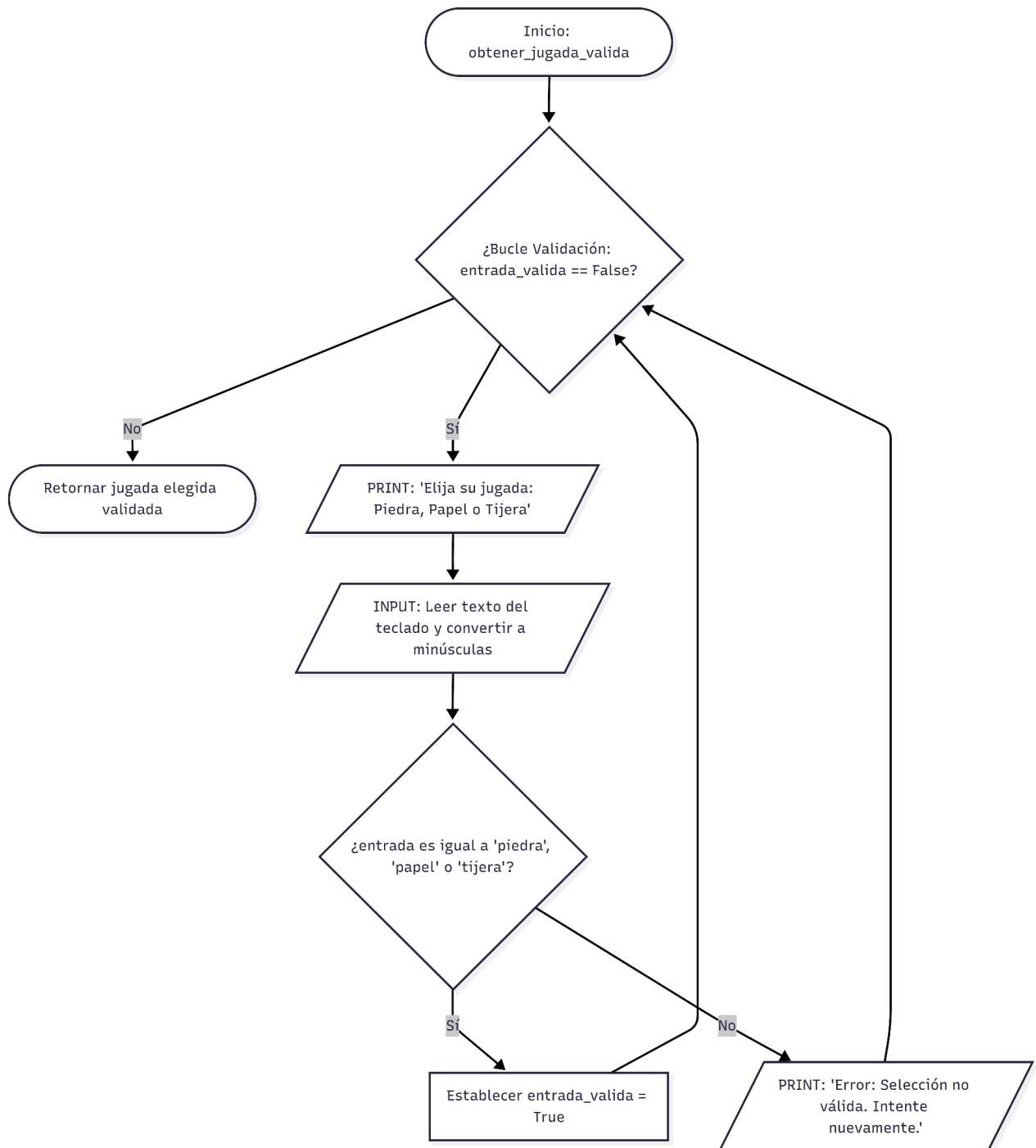
Con el objetivo de solventar de forma absoluta y técnica las observaciones respecto al acoplamiento de código y la falta de diagramación de funciones específicas, el flujo de control del software se ha descompuesto rigurosamente en tres flujoramas independientes y especializados, los cuales reflejan con precisión matemática las operaciones implementadas en el código fuente:

1. Controlador Principal del Sistema : Governa el ciclo de vida de la aplicación. Ilustra de manera exacta la instanciación de los objetos principales del modelo y la vista, la captura inicial del nombre y el control del bucle de sesión activa mediante una condición repetitiva. Este flujograma detalla la evaluación jerárquica de la opción ingresada en el menú por el operario, derivando el flujo sincronamente hacia la validación de jugadas, el reinicio de contadores o el cierre ordenado del software.

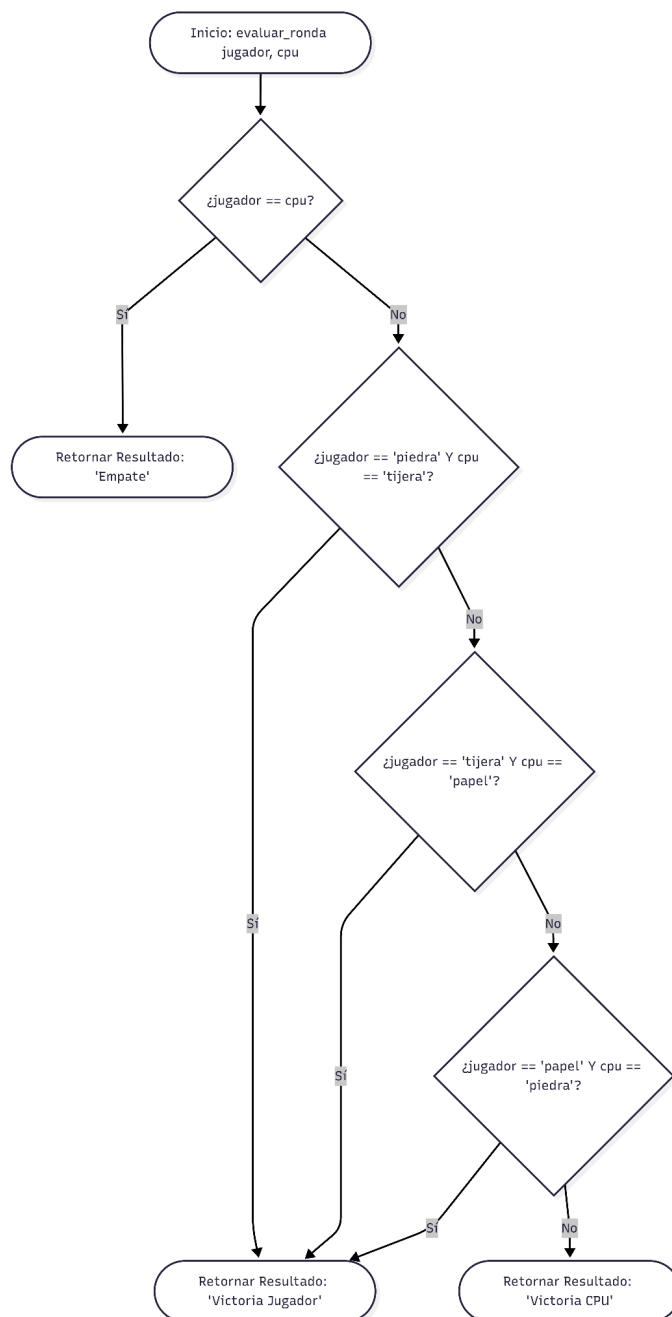


REINVENTEMOS
EL FUTURO

2. Bucle de Validación Controlada en la Vista: Mapea el método de captura de datos de la interfaz. Representa explícitamente las instrucciones de salida de pantalla (PRINT) y de lectura de datos periféricos (INPUT). A través de un bucle de control interno condicionado por un estado booleano, describe cómo el sistema evalúa la validez de la entrada del usuario, forzando la repetición del ciclo ante entradas erróneas y garantizando que solo fluyan datos lícitos (piedra, papel o tijera) hacia las capas lógicas internas del dominio.



3. Árbol Determinista de Decisiones de Reglas de Negocio: Corresponde al núcleo algorítmico del motor de juego. Detalla de forma minuciosa la evaluación de las combinaciones de juego. Inicialmente, procesa una condicional de igualdad para dictaminar estados de empate. Al detectar asimetría en las jugadas, el flujo avanza secuencialmente a través de un árbol de decisión compuesto por tres condicionales compuestas unidas por operadores lógicos booleanos, las cuales validan explícitamente los tres escenarios exclusivos de victoria del jugador humano: piedra vence a tijera, tijera vence a papel y papel vence a piedra. De no cumplirse ninguna condición, el flujo se bifurca de manera determinista hacia el retorno de victoria de la CPU, cerrando el ciclo operativo de la ronda con absoluta precisión.



REINVENTEMOS
EL FUTURO

5. REFLEXIÓN SOBRE EL IMPACTO DE LA TECNOLOGÍA EN LA SOCIEDAD

En concordancia con el macro-tema integrador definido para el presente curso, la incorporación exponencial de las nuevas tecnologías informáticas e industriales en la sociedad contemporánea ha reconfigurado radicalmente las estructuras socioculturales, los hábitos de comunicación y los paradigmas de resolución de problemas lógicos a nivel global. La transición hacia un entorno enteramente digitalizado e interconectado implica que actividades cotidianas, recreativas y estratégicas sean paulatinamente absorbidas y gestionadas por sistemas de software automatizados. Esta digitalización democratiza el acceso a herramientas educativas y recreativas avanzadas, permitiendo que plataformas interactivas operen de manera continua y autónoma las 24 horas del día, eliminando barreras geográficas y optimizando los flujos de interacción humana.

Sin embargo, esta profunda influencia socio-tecnológica deposita una inmensa responsabilidad técnica y ética sobre los hombros de los futuros ingenieros en Inteligencia Artificial y tecnologías emergentes. El desarrollo de cualquier software interactivo, incluso aquellos enmarcados en el ámbito lúdico o de entretenimiento, debe regirse por los principios fundamentales de transparencia algorítmica, equidad matemática, accesibilidad universal y seguridad en el manejo del estado y persistencia de datos. Al automatizar procesos de selección o evaluación mediante modelos computacionales, es imperativo garantizar la neutralidad y la auditabilidad de los algoritmos; por ejemplo, asegurar que un oponente artificial actúe bajo estrictas leyes de probabilidad uniforme e incorruptible, mitigando sesgos o manipulaciones que quiebren la confianza del usuario dentro del ecosistema digital. Así, la ingeniería de software se consolida no solo como una disciplina técnica abstracta, sino como un agente ético de cambio social enfocado en construir un futuro tecnológico justo, transparente y beneficioso para la colectividad.

6. CONCLUSIONES Y LOGROS RELACIONADOS

La finalización e integración del presente proyecto informático de desarrollo permite extraer las siguientes conclusiones fundamentales y basadas firmemente en los resultados empíricos obtenidos por la solución algorítmica:

- Se validó de manera exitosa la viabilidad práctica del patrón de diseño arquitectónico Modelo-Vista-Controlador (MVC), logrando un desacoplamiento absoluto de capas que independiza las reglas lógicas del negocio de los canales físicos de entrada/salida de datos periféricos.

REINVENTEMOS
EL FUTURO

- La simulación algorítmica del oponente de la CPU demostró un estricto apego a los postulados teóricos del Equilibrio de Nash, garantizando aleatoriedad pura a través de una distribución probabilística uniforme simétrica que imposibilita la explotación táctica del sistema por parte del usuario.
- El control de ciclos estructurados mediante el bucle principal demostró una eficiencia temporal óptima, arrojando tiempos de procesamiento y respuesta imperceptibles que no superan el umbral crítico de los 100 milisegundos bajo condiciones normales de memoria local.

7. IMPLICACIONES Y LIMITACIONES DEL PROYECTO

Con una perspectiva técnica analítica, transparente y autocrítica, es mandatorio detallar las implicaciones operativas y limitaciones reales evidenciadas durante el ciclo de desarrollo e implementación de la solución informática:

- Limitación de Persistencia de Datos Volátil: El estado mutatorio de las partidas de juego y los puntajes acumulados en el marcador se gestionan de forma exclusiva en la memoria RAM del sistema local. Esto implica que la interrupción física o cierre del proceso informático deriva en la pérdida total e irreversible del historial de la sesión, denotando la ausencia de una capa de persistencia persistente no volátil (bases de datos estructuradas o persistencia física de archivos locales).
- Limitación de Interfaz de Usuario Restringida: La interacción del sistema operativo se encuentra confinada a un entorno de interfaz de línea de comandos (CLI) basado puramente en texto, limitando la accesibilidad ergonómica para usuarios no técnicos habituados a entornos visuales interactivos contemporáneos.
- Implicación de Escalabilidad y Proyección Industrial: Gracias al estricto aislamiento arquitectónico implementado desde la raíz del código, las limitaciones previas actúan como ventanas de oportunidad controladas para futuras fases de expansión. La Capa de Dominio (Modelo matemático probabilístico) se encuentra 100% preparada para acoplar conectores de bases de datos relacionales en la nube e interfaces gráficas avanzadas en entornos web o móviles sin requerir ningún tipo de refactorización o alteración en la lógica central de negocio existente.

8. REFERENCIAS BIBLIOGRÁFICAS

Gibbons, R. (1992). *Un primer curso de teoría de juegos*. Antoni Bosch Editor.

Larman, C. (2003). *UML y patrones: Una introducción al análisis y diseño orientado a objetos y al proceso unificado* (2.a ed.). Prentice Hall.

Nash, J. (1950). Equilibrium points in n-person games. *Proceedings of the National Academy of Sciences*, 36(1), 48-49.

Pressman, R. S. (2010). *Ingeniería del software: un enfoque práctico* (7.a ed.). McGraw-Hill.

Sommerville, I. (2011). *Ingeniería de software* (9.a ed.). Addison-Wesley.